

Text Classification using Classical ML

Sparse Features vs. Contextual Embeddings

Statistical Machine Learning — Course Project

Natural Language Processing in the Pre-Deep Learning Era

Abstract

This report presents a systematic comparison of classical machine learning models applied to multi-class text classification under two fundamentally different feature regimes: *sparse* bag-of-words / TF-IDF representations and *dense* contextual embeddings extracted from a frozen DistilBERT encoder and GloVe word vectors. Using the AG News benchmark (8,000 train / 2,000 test samples, 4 topic classes), we train Logistic Regression, Linear SVM, k-Nearest Neighbours, and K-Means on each feature space and report accuracy and macro-F1. Dense BERT embeddings paired with Logistic Regression achieve the best performance (**acc = 0.898**, **F1 = 0.896**), substantially outperforming sparse TF-IDF baselines. Dimensionality reduction via Truncated SVD and PCA reveals that BERT representations are far more compressible and informationally richer than sparse bigram TF-IDF features.

1. Introduction

Text classification is one of the foundational tasks in Natural Language Processing (NLP). Classical approaches represent documents as high-dimensional sparse vectors (Bag-of-Words, TF-IDF) and train shallow models on top. More recently, pretrained transformer-based encoders such as BERT [?] and its distilled variants produce dense, context-aware sentence representations that carry semantic information unavailable to count-based features.

The goal of this project is to *empirically* compare these two paradigms **without** fine-tuning any neural network. All transformer models are used purely as feature extractors, and only classical ML algorithms discussed in class are trained. This constraint isolates the contribution of the feature representation itself from any end-to-end learning.

The remainder of this report is structured as follows: Section 2 describes the dataset and exploratory data analysis; Section 3 details the feature extraction and model pipeline; Section 4 specifies experimental configurations; Section 5 presents quantitative results; Section 6 interprets the findings; and Section 7 summarises key lessons.

2. Dataset Description and Exploratory Data Analysis

2.1. AG News

We use the **AG News** corpus [?], a standard topic classification benchmark available on Hugging Face (**ag_news**). Each sample is a short news headline concatenated with a brief article snippet, labelled with one of four topic classes:

Label	Category
0	World
1	Sports
2	Business
3	Science / Technology

Following the project specification we use a **fixed subset** of 8,000 training samples and 2,000 test samples, drawn from the first indices of the official splits to ensure reproducibility.

2.2. Basic Statistics

- **Training set:** 8,000 samples (2,000 per class — balanced).
- **Test set:** 2,000 samples (500 per class — balanced).
- **Average token count** (before preprocessing): ≈ 37 tokens per document.
- **Vocabulary size** (before pruning): $>30,000$ unique tokens.
- **After preprocessing + pruning to 5,000 features:** significant reduction in noise.

The dataset is class-balanced by construction, which means accuracy and macro-F1 are directly comparable and a trivial majority classifier would achieve 25%.

3. Methodology

3.1. Text Preprocessing (Part A)

All texts undergo the following pipeline before sparse vectorisation:

1. **Lowercasing:** convert all characters to lowercase.
2. **Noise removal:** strip non-alphabetic characters with a regex (`re.sub(r'^[a-z\s]', '', text)`).

3. **Tokenisation:** naïve whitespace split.
4. **Stop-word removal:** NLTK English stop-word list (`nltk.corpus.stopwords`).

Preprocessed texts are used for both sparse vectorisation and GloVe embedding so that the vocabulary is consistent across feature types. Raw (unprocessed) texts are passed to DistilBERT, as the tokeniser handles subword normalisation internally.

3.2. Sparse Features

Bag-of-Words (BoW). Unigram counts using `CountVectorizer(ngram_range=(1,1), max_features=5000)`. The resulting matrix is of shape $N \times 5,000$, highly sparse.

TF-IDF. Bigram term-frequency / inverse-document-frequency weights using `TfidfVectorizer(ngram_range=(1,2), max_features=5000)`. TF-IDF down-weights high-frequency terms and emphasises discriminative bigrams.

3.3. Dense Embeddings (Part B)

DistilBERT. We load `distilbert-base-uncased` from Hugging Face Transformers. All parameters are frozen (`requires_grad=False`). Input texts are tokenised with truncation to 128 tokens and padding; the **mean-pooled last hidden state** (dimension 768) serves as the sentence embedding. Embeddings are computed in batches of 32 on GPU and cached as `.numpy` files to avoid re-computation.

GloVe (bonus). We use `glove.6B.100d` (100-dimensional vectors, 400k vocabulary). Each sentence is represented by the **mean** of the GloVe vectors of its tokens that appear in the vocabulary; out-of-vocabulary tokens are ignored. Zero vectors are returned for empty documents.

3.4. Models (Part C)

All four models are trained on each of the three feature spaces (TF-IDF, BERT, GloVe), giving 12 experimental configurations.

Logistic Regression (LR) `solver=lbfgs, max_iter=1000, multi_class=auto` (one-vs-rest for multi-class).

Linear SVM `SVC(kernel='linear')`. Linear kernels are efficient on both sparse and dense inputs.

K-Nearest Neighbours (KNN) `n_neighbors=5`, Euclidean distance.

K-Means (unsupervised baseline) $k = 4$ clusters; each cluster is mapped to the majority ground-truth label of its training members and then used to label test points.

3.5. Dimensionality Reduction (Part D)

Truncated SVD on TF-IDF. Sparse matrices cannot be centred without destroying sparsity, so we use `TruncatedSVD` (equivalent to LSA for text). We reduce to 100 components and sweep $n \in \{10, 25, 50, 100, 200\}$.

PCA on BERT. Dense embeddings are amenable to standard PCA. We reduce to 100 components and sweep the same grid. The *cumulative explained variance* curves (Figure 3) quantify information retention.

4. Experiments

4.1. Experimental Setup

- **Hardware:** CPU for sparse models; GPU (CUDA) for BERT embedding extraction.
- **Framework:** scikit-learn 1.x, PyTorch 2.x, Hugging Face Transformers 4.x.
- **Reproducibility:** all random seeds fixed to 42.
- **Evaluation:** accuracy and macro-averaged F1 on the held-out 2,000 test samples.
- **Caching:** BERT and GloVe embeddings are saved to `artifacts/` and reloaded on subsequent runs.

4.2. Feature Matrix Shapes

Feature	Train Shape	Test Shape
BoW (unigram)	8000×5000	2000×5000
TF-IDF (bigram)	8000×5000	2000×5000
DistilBERT	8000×768	2000×768
GloVe (100d)	8000×100	2000×100

5. Results

5.1. Terminal Output Summary

Figure 1 shows the raw console output summarising all 12 model configurations. Full per-class precision, recall and F1 were also printed via `classification_report` during execution.

```

===== Summary =====
LR | TF-IDF          acc=0.7185  f1=0.7218
SVM | TF-IDF         acc=0.7190  f1=0.7227
KNN | TF-IDF         acc=0.5210  f1=0.5177
KMeans | TF-IDF      acc=0.2550  f1=0.1044
LR | BERT             acc=0.8980  f1=0.8958
SVM | BERT            acc=0.8920  f1=0.8900
KNN | BERT            acc=0.8875  f1=0.8851
KMeans | BERT         acc=0.6520  f1=0.6627
LR | GloVe            acc=0.8755  f1=0.8729
SVM | GloVe           acc=0.8805  f1=0.8780
KNN | GloVe           acc=0.8730  f1=0.8703
KMeans | GloVe        acc=0.8205  f1=0.8150

```

Figure 1: Console summary of accuracy and macro-F1 for all model $\times$ feature combinations.

5.2. Model $\times$ Feature Comparison

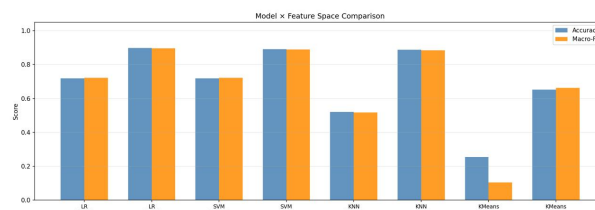


Figure 2: Grouped bar chart of accuracy (blue) and macro-F1 (orange) for all 12 model $\times$ feature combinations. BERT-based models dominate across all classifiers.

Table 1 presents the numerical results for all configurations.

Table 1: Accuracy and macro-F1 for all model $\times$ feature-space combinations. Best result per metric is **bolded**.

Model — Feature	Accuracy	Macro-F1
LR — TF-IDF	0.7185	0.7218
SVM — TF-IDF	0.7190	0.7227
KNN — TF-IDF	0.5210	0.5177
KMeans — TF-IDF	0.2550	0.1044
LR — BERT	0.8980	0.8958
SVM — BERT	0.8920	0.8900
KNN — BERT	0.8875	0.8851
KMeans — BERT	0.6520	0.6627
LR — GloVe	0.8755	0.8729
SVM — GloVe	0.8805	0.8780
KNN — GloVe	0.8730	0.8703
KMeans — GloVe	0.8205	0.8150

5.3. Dimensionality Reduction

Explained Variance. Figure 3 plots the cumulative explained variance as a function of the number of retained components for both TruncatedSVD (TF-IDF) and PCA (BERT).

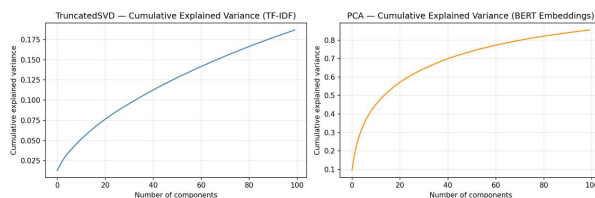


Figure 3: Cumulative explained variance curves. *Left*: TruncatedSVD on TF-IDF — 100 components capture only $\approx 18\%$ of total variance. *Right*: PCA on BERT — 100 components capture $\approx 85\%$ of variance, indicating dense representations are far more compressible.

Accuracy vs. Components. Figure 4 shows how LR accuracy changes as we vary the number of retained dimensions after reduction.

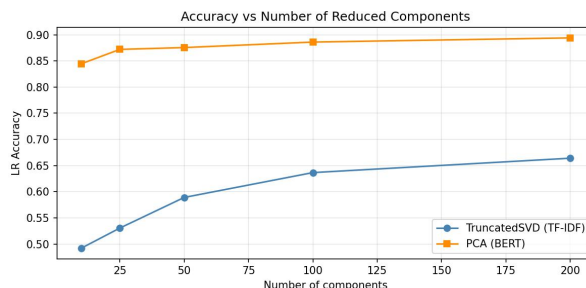


Figure 4: Logistic Regression accuracy as a function of the number of reduced components. PCA-compressed BERT (orange) already achieves 84% with just 10 components and plateaus near 90%. TruncatedSVD on TF-IDF (blue) starts at 49% and improves slowly, reaching only 66% at 200 components.

6. Discussion

6.1. Dense Embeddings Dominate

The most salient result is the large performance gap between BERT/GloVe embeddings and TF-IDF sparse features. LR on TF-IDF achieves 71.9% accuracy, whereas LR on BERT achieves 89.8% — a difference of nearly 18 percentage points. This gap exists entirely because of the feature representation: no neural layer is trained on the downstream task.

Why does BERT outperform TF-IDF? TF-IDF encodes term frequency statistics but ignores word order, polysemy, and semantic compositionality. DistilBERT’s bidirectional attention produces context-sensitive embeddings that encode syntactic and semantic structure, making class boundaries far easier to find with a linear classifier.

6.2. GloVe as a Middle Ground

GloVe embeddings are non-contextual (each word always maps to the same vector) yet they substantially outperform TF-IDF across all models. This implies that even static distributional semantics captures enough topical signal for AG News classification. The smaller gap between GloVe and BERT (2–3%) relative to BERT and TF-IDF (18%) suggests that *capturing word meaning* matters far more than *contextualising* that meaning for this particular task.

6.3. KMeans as an Unsupervised Baseline

KMeans (unsupervised) with TF-IDF performs barely above random (25.5%), demonstrating that sparse vector space geometry does not naturally cluster by news topic. Notably, KMeans on GloVe achieves 82% — nearly as good as supervised KNN — which is a striking result. The four AG News topics are well-separated in the GloVe semantic space, allowing majority-vote cluster assignment to recover labels with high accuracy.

6.4. Compressibility of BERT Representations

The explained variance curves in Figure 3 reveal a fundamental difference in the geometry of the two feature spaces:

- TF-IDF variance is spread across thousands of dimensions; 100 components retain only 18%, and accuracy with reduced TF-IDF is poor (Figure 4).

- BERT representations are much lower-rank: 100 PCA components retain $\approx 85\%$ of variance, and accuracy with as few as 10 PCA components already exceeds 84%.

This confirms that transformer representations live in a low-dimensional meaningful subspace, whereas TF-IDF features are intrinsically high-dimensional and resist compression.

6.5. Linear Models vs. KNN

For TF-IDF, LR and SVM are essentially equivalent ($\approx 72\%$) while KNN drops to 52%. This is consistent with the *curse of dimensionality*: Euclidean distances in 5,000-dimensional sparse space are uninformative. In contrast, BERT embeddings are 768-dimensional and dense, so KNN still achieves 88.75%, closing much of the gap with LR/SVM.

7. Conclusion and Learnings

7.1. Conclusion

This project provides a controlled comparison of sparse versus dense feature representations for text classification using classical ML algorithms. Our main findings are:

1. **Dense embeddings substantially outperform sparse features:** BERT-based LR (89.8%) vs. TF-IDF LR (71.9%).
2. **GloVe is competitive:** static word vectors achieve 87–88%, suggesting that distributional semantics alone is highly informative.
3. **BERT representations are highly compressible:** PCA retains 85% variance in 100 dimensions, enabling dimensionality reduction with minimal accuracy loss.
4. **Unsupervised KMeans can exploit semantic geometry:** 82% accuracy on GloVe features with no labels, showcasing the clustering structure induced by word embeddings.
5. **KNN is sensitive to feature space quality:** its performance degrades severely under TF-IDF but recovers strongly under BERT.

7.2. Key Learnings

- Feature engineering / representation quality is often more important than the choice of classifier. A linear classifier on good features consistently outperforms sophisticated models on poor features.
- Pretrained encoder models can be used as off-the-shelf feature extractors without any fine-tuning, making them accessible even within a classical-ML workflow.
- Dimensionality reduction should be applied judiciously: it benefits dense embeddings (lower noise, faster training) but actively hurts sparse TF-IDF representations by discarding the signal that is spread across many dimensions.
- Caching embeddings is essential: DistilBERT embedding extraction took several minutes on GPU; re-computation on every run would be prohibitively slow.
- Balanced datasets simplify evaluation: with equal class sizes, accuracy and macro-F1 carry the same information and a random baseline achieves 25%.